Universidad de Deusto
Deustuko Unibertsitatea
University of Deusto

Seguridad de la Información
Honeypot

Grupo D
Iker Aramendi
Joseba Eceiza
Jon Larrañaga

1. Introducción **3**
a. Descripción 3
b. Objetivo del proyecto 3
c. Herramientas utilizadas 4
2. Despliegue del sistema **4**
3. Recolección de datos **5**
4. Procesamiento y feature engineering **5**
5. Modelo machine learning **6**
6. Evaluación resultados **6**
7. Problemas y limitaciones **7**
8. Resultados obtenidos **8**

1. Introducción

a. Descripción

Un honeypot es un sistema informático diseñado para simular vulnerabilidades o servicios expuestos con el objetivo de atraer atacantes.

En lugar de proteger directamente sistemas reales, los honeypots se usan para:

- Observar comportamiento de atacantes
- Capturar Malware
- Analizar técnicas de intrusión
- Generar datasets de ataques

Existen varios tipos de honeypots

- **Honeypots de baja interacción (nuestro caso)**
  - Simulan servicios pero no ejecutan un sistema real, por ejemplo: **Cowrie**
    - Ventajas→ Seguros, fáciles de instalar y bajo consumo
    - Desventajas → Menos realistas
- **Honeypots de alta interacción**
  - Son sistemas reales vulnerables, por ejemplo: máquinas Linux reales.
    - Ventajas → Comportamiento muy real
    - Desventajas→ Peligrosos, complejos
- **Honeypots híbridos**
  - Plataformas que integran varios honeypots, por ejemplo: T-Pot. Incluye SSH honeypots, web honeypots y malware capture

b. Objetivo del proyecto

El objetivo principal de este proyecto es desplegar un honeypot en un entorno en la nube para capturar actividad maliciosa real y, posteriormente, analizarla mediante técnicas de machine learning.

De forma más específica, se busca:

- Desplegar un honeypot (Cowrie) en AWS para simular un sistema vulnerable expuesto a Internet.
- Recopilar y almacenar logs de ataques reales en formato estructurado (JSON).

- Transformar los logs en un dataset organizado por sesiones mediante feature engineering.
- Aplicar técnicas de machine learning para clasificar distintos tipos de ataques.
- Evaluar el rendimiento del modelo mediante métricas como accuracy, precision, recall y F1-score.

c. Herramientas utilizadas

- **Amazon Web Services (AWS):** plataforma en la nube utilizada para desplegar el honeypot en un entorno accesible desde Internet.

- **Cowrie:** honeypot de media interacción que simula servicios SSH/Telnet y registra actividad de atacantes.

- **Python:** lenguaje utilizado para el procesamiento de datos y el desarrollo del modelo de machine learning.

- **Pandas:** librería para la manipulación y transformación de los logs en un dataset estructurado.

- **Scikit-learn:** librería utilizada para la creación, entrenamiento y evaluación de los modelos de clasificación.

- **Jupyter Notebook:** entorno de desarrollo utilizado para el análisis exploratorio y la implementación del pipeline.

- **GeoIP:** Base de datos usado para mapear la IP con el país.

2. Despliegue del sistema

El honeypot Cowrie fue desplegado en una instancia de Amazon Web Services (AWS) utilizando como sistema operativo Ubuntu, debido a su estabilidad y amplio uso en entornos de servidores.

Para la exposición del servicio, se configuró el acceso SSH en el puerto 2222, en lugar del puerto estándar 22, con el objetivo de evitar conflictos con servicios del sistema y facilitar la monitorización del honeypot. El acceso a Telnet también se habilitó en el puerto 2323, en lugar del puerto 23. Para llevar a cabo estas configuraciones, se tuvieron que modificar las reglas de entrada de AWS.

Una vez configurado, Cowrie se expuso a Internet simulando un sistema Linux vulnerable, permitiendo registrar intentos de acceso no autorizados, comandos ejecutados y sesiones completas de los atacantes

3. Recolección de datos

La recolección de datos se realizó a través de los logs generados (formato JSON) por Cowrie durante su ejecución en la instancia de AWS. Cada interacción con el honeypot se registra automáticamente, incluyendo intentos de inicio de sesión, comandos ejecutados, direcciones IP de origen y eventos de sesión.

Los datos se almacenaron en formato JSON, organizados por fechas, por lo que fue necesario unir múltiples archivos para obtener un único conjunto de datos completo.

La descarga de logs se hizo levantando un servidor con python mediante el comando *sudo python3 -m http.server 80*.

Finalmente, los datos combinados fueron transferidos al entorno local para su posterior análisis y transformación en un dataset estructurado por el código de sesión, listo para su uso.

4. Procesamiento y feature engineering

Una vez recopilados los logs del honeypot, se realizó un proceso de limpieza y transformación de los datos para convertirlos en un formato adecuado para machine learning.

En primer lugar, los datos fueron agrupados por sesión, de forma que cada registro representa la actividad completa de un atacante durante su interacción con el sistema. Esto permitió pasar de eventos individuales a un nivel de análisis basado en comportamiento.

Durante el análisis inicial se identificó un problema importante: la mayoría de las sesiones no contenían ejecución de comandos, limitándose únicamente a intentos de autenticación o escaneo inicial. Esto reducía la información disponible para diferenciar tipos de ataques más complejos.

Para solventarlo, se decidió enriquecer el dataset mediante la creación de variables derivadas, asignando etiquetas de tipo de ataque en función del comportamiento observado (por ejemplo, fuerza bruta, reconocimiento, actividad de bot o sesiones interactivas).

A partir de cada sesión se generaron múltiples características (features), incluyendo:

- Métricas de autenticación (intentos de login, éxito, ratio de fallos).
- Actividad del atacante (número de comandos, comandos únicos, longitud media).

- Indicadores de comportamiento como reconocimiento del sistema, descargas o ejecución de comandos.
- Variables temporales como duración de la sesión y comandos por segundo.

Este enfoque permitió transformar datos poco estructurados y en muchos casos incompletos en un dataset más informativo, capaz de representar distintos patrones de ataque de forma diferenciada.

Finalmente, el resultado es un dataset estructurado por sesiones, donde cada fila representa el comportamiento agregado de un atacante y cada columna una característica relevante para su clasificación.

5. Modelo machine learning

Una vez construido el dataset estructurado por sesiones, se aplicaron técnicas de machine learning para la clasificación de los distintos tipos de ataque detectados en el honeypot.

Dado que una misma sesión puede presentar múltiples comportamientos (por ejemplo, reconocimiento y fuerza bruta a la vez), el problema se formuló como una clasificación multilabel.

Para ello, las etiquetas de ataque fueron transformadas en formato binario mediante, “MultiLabelBinarizer” permitiendo representar cada clase como una variable independiente.

Posteriormente, se seleccionó el algoritmo Random Forest, debido a su buen rendimiento en problemas con datos tabulares, su capacidad para manejar relaciones no lineales y su robustez frente a ruido y desbalance de clases.

El modelo se entrenó utilizando una división train-test split, separando el conjunto de datos en entrenamiento y validación para evaluar su capacidad de generalización.

Además, se utilizó (class_weight = “balanced”) con el objetivo de mitigar el desbalance entre clases, ya que algunas categorías de ataque estaban mucho más representadas que otras.

El modelo final fue evaluado mediante métricas estándar de clasificación multilabel, como precisión, recall y F1-score, obteniendo mejores resultados en las clases mayoritarias y un rendimiento más limitado en aquellas con menor representación.

6. Evaluación resultados

El rendimiento del modelo se evaluó utilizando un conjunto de prueba independiente, aplicando métricas estándar de clasificación multilabel como

precisión, recall y F1-score, además de sus promedios (micro, macro y weighted average).

Los resultados obtenidos muestran un rendimiento global elevado en términos de micro y weighted average, lo que indica que el modelo es capaz de clasificar correctamente la mayoría de las sesiones, especialmente aquellas pertenecientes a las clases más frecuentes del dataset.

Sin embargo, al analizar el macro average, se observa un rendimiento significativamente inferior. Esto se debe al fuerte desbalance de clases, ya que algunas categorías de ataque están muy representadas (como sesiones de escaneo o autenticación sin acción), mientras que otras cuentan con muy pocos ejemplos o incluso ninguno en el conjunto de test.

En las clases mayoritarias, el modelo obtiene valores cercanos a 1 en precisión, recall y F1-score, lo que indica una buena capacidad de detección de patrones comunes. En cambio, en las clases minoritarias el rendimiento es más inestable o directamente nulo, debido a la escasez de muestras.

Este comportamiento es típico en problemas de ciberseguridad reales, donde ciertos tipos de ataques son mucho más frecuentes que otros y donde muchos de los ataques no contienen comandos, generando datasets altamente desbalanceados.

En general, los resultados indican que el modelo es efectivo para la detección de patrones principales de actividad maliciosa, aunque su capacidad para generalizar a clases menos representadas es limitada.

7. Problemas y limitaciones

Durante el desarrollo del proyecto se han identificado varias limitaciones inherentes tanto a la naturaleza del dataset como al entorno de captura de datos.

En primer lugar, el dataset presenta un fuerte desbalance de clases, ya que la mayoría de las sesiones corresponden a comportamientos de escaneo o intentos de autenticación, mientras que otros tipos de ataques aparecen con muy pocas muestras o incluso de forma residual. Esto afecta directamente al rendimiento del modelo en clases minoritarias.

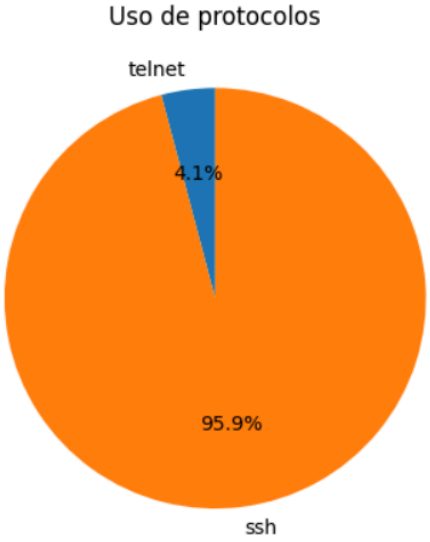
En segundo lugar, se observa una limitación en la riqueza de las sesiones, ya que gran parte de los ataques no incluyen ejecución de comandos, lo que reduce la cantidad de información disponible para diferenciar comportamientos más complejos.

Otra limitación importante es el uso de etiquetado heurístico, ya que las clases de ataque se han generado a partir de reglas basadas en el comportamiento observado, en lugar de contar con etiquetas reales verificadas (ground truth). Esto puede introducir sesgos en el aprendizaje del modelo.

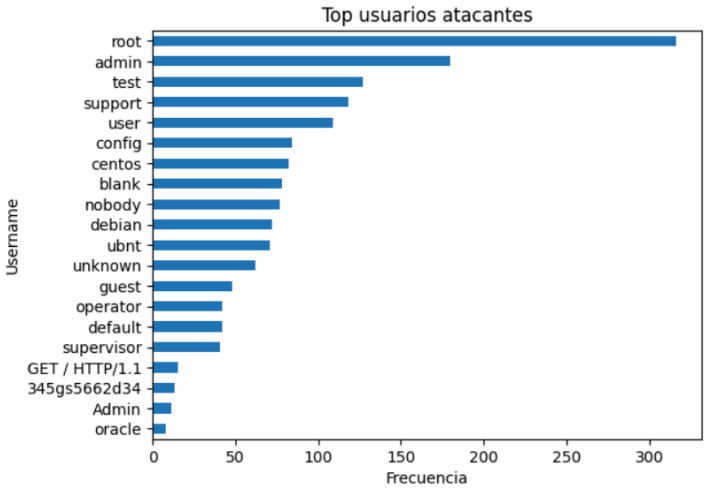
Finalmente, al tratarse de un entorno controlado mediante un honeypot, los ataques capturados pueden no representar completamente la diversidad de amenazas presentes en entornos de producción reales.

8. Resultados obtenidos

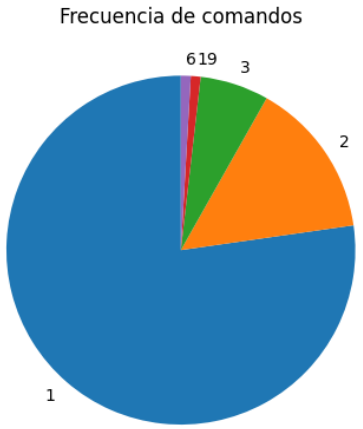
Había puertos habilitados:
Puerto 2323 para Telnet
Puerto 2222 para SSH



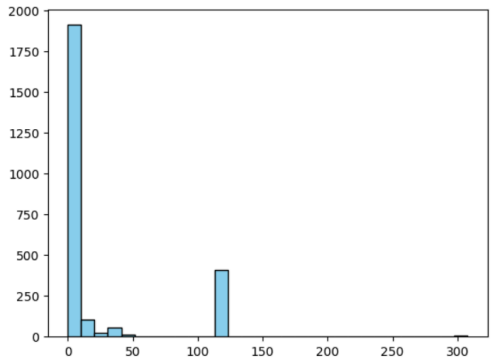
En la gráfica observamos que la mayoría de los accesos se han hecho con ssh.

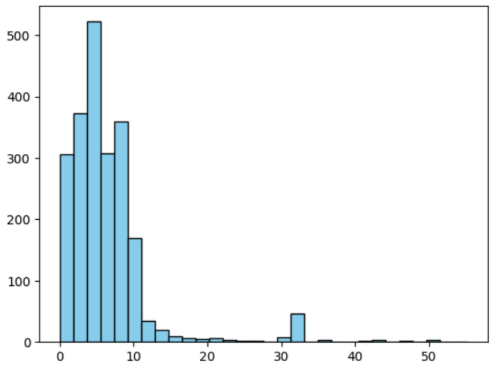


Vemos que los nombres de usuario más habituales son root admin y test.



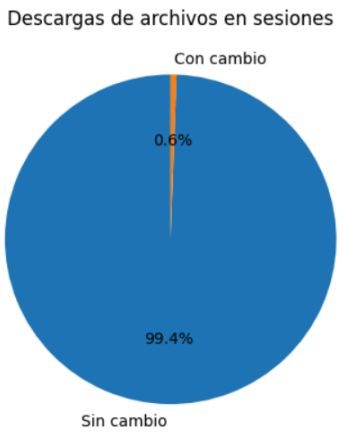
El 4,2% de las sesiones contiene algún comando.
El 77,3% de las sesiones que han tenido algún comando han ejecutado un solo comando. El 14.5% han usado 2 comandos en una sesión, el 6.4% 3 comandos, el 0.9% 19 comandos y el 0.9% 6 comandos.

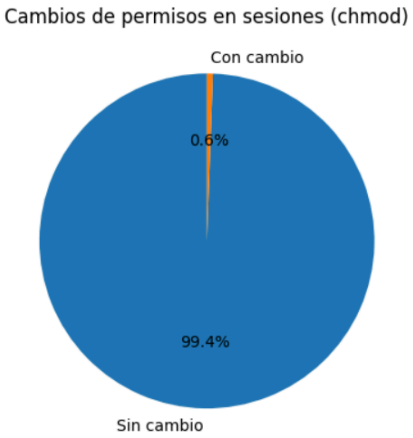




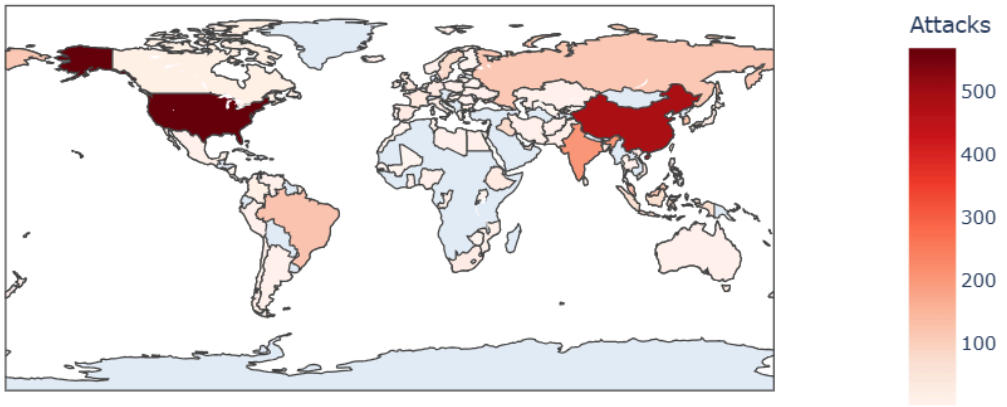
Mediante está gráfica observamos la distribución del tiempo de la duración de las sesiones. Observamos que la mayoría de las sesiones tienen una duración menor de 30 segundos.

Si ampliamos la gráfica en los primeros 60 segundos, podemos observar que la mayoría de las sesiones han tenido una duración menor que 10 segundos.

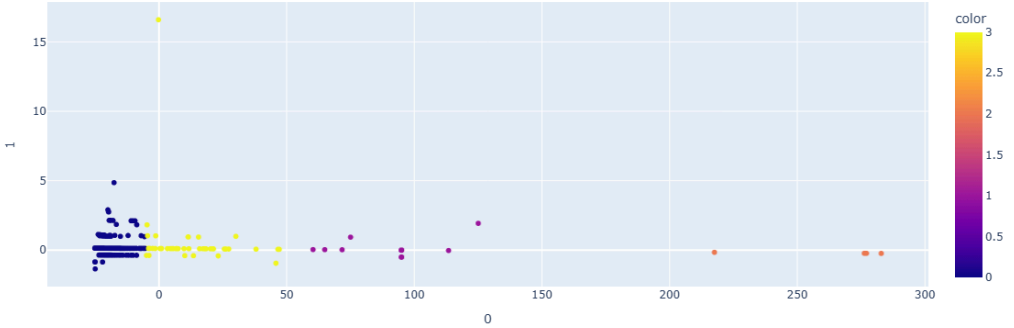




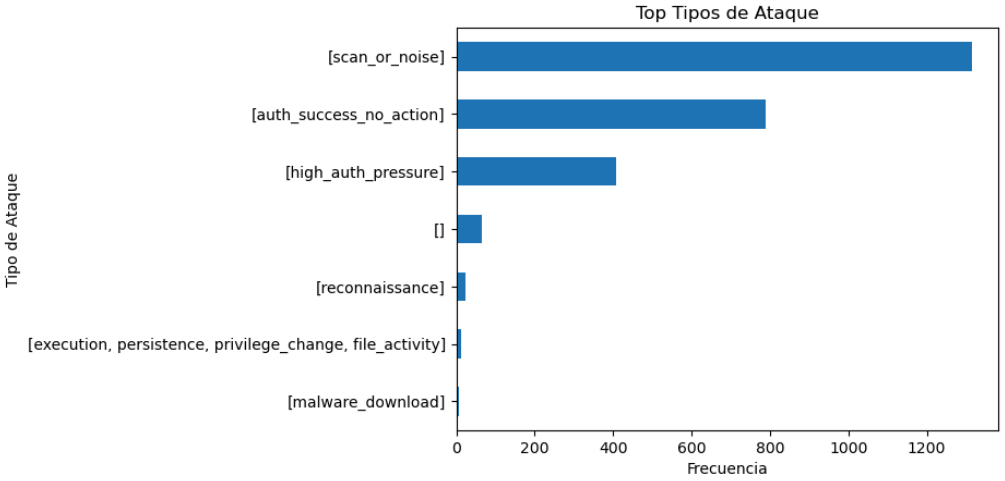
El 0,6% de las sesiones han sufrido algún cambio de permisos y descarga de algún archivo, ambos han ocurrido en las mismas sesiones.



Al convertir las ips de las sesiones en países y coloreamos los paises mas frecuentes en colores más oscuros, vemos que china y estados unidos son los paises que mas sesiones han tenido.



PCA: Al visualizar los primeros dos componentes y darles el color que el algoritmo kmeans haya puesto en cada sesión, vemos que hay una gran concentración de sesiones que se parecen. Esto se da porque las sesiones suelen usar un ataque de gran número de peticiones.



Finalmente, este es el top 7 tipos de ataque que hemos percibido.